Estuin Bobadilla-Cruz
Basic Alg HW1

Q1)

9/13

Q1) Consider the following recursion w. $a_1 = 3, a_2 = 5, \forall n \geq 2$. (n is an integer)

$$\underset{\text{Next}}{a_{n+1}} = \underset{\text{current}}{3a_n} - \underset{\text{prev}}{2a_{n-1}}$$

Now, prove $[a_n = 2^n + 1]$ using strong Induction

Proof by Strong Induction

Base Case(s)

Case 1: Let $n = 1$ thus using $a_n = 2^n + 1$, $a_1 = 2^1 + 1 = 3$ ✓
base case holds true.

Case 2: Let $n = 2$. Thus $a_2 = 2^2 + 1 = 4 + 1 = 5$ ✓ Base case holds.

Thus both of our base cases when $n = 1, 2$ hold true.

Inductive step:

(i) Assumption: Assume $a_n = 2^n + 1$ is true for $\forall n$, $n = 1, \ldots, k$
Such that $a_k = 2^k + 1$ and $a_{k-1} = 2^{k-1} + 1$

Assuming $a(k)$, $a(k-1)$ are true, $a(k+1)$ is also true. By the recursion relation and inductive hypothesis, we have

$$a_{k+1} = 3a_k - 2a_{k-1}$$
$$a_k = 2^k + 1$$
$$a_{k-1} = 2^{k-1} + 1$$

Observe that through substitution,

$$a_{k+1} = 3(2^k + 1) - 2(2^{k-1} + 1)$$

using Exponent Laws $2 \cdot 2^{k-1} = 2^k$ (same base)

By Distribution,

$$a_{k+1} = 3 \cdot 2^k + 3 - 2 \cdot 2^{k-1} + 2$$
$$= 3 \cdot 2^k + 3 - (2^k + 2)$$
$$= 3 \cdot 2^k - 2^k + 3 - 2$$

add powers

$$= 3 \cdot 2^k - (1)2^k + 1 = 2 \cdot 2^k + 1 = 2^{k+1} + 1 \checkmark$$

Inductive Conclusion: a_n is true $\forall n \geq 2$, $n \in \mathbb{N}$ Δ using Strong Induction

Estela Bobadilla-Cruz Q2)

Rank the following functions in order of their Asymptotic growth such that $f1 = O(f2)$, $f2 = O(f3)$, and so on

Note $f1 = O(f2)$ means the growth rate of $f1$ is less than or equal to the growth rate of $f2$

Order / Ranking

e) $2^{\log_2 n}$

b) $n^{1.5} \log_2 n$

c) $n^2 - 1$

a) 2^n

f) 3^n

d) $n!$

Ezun Berz Q3)

c) If $f = O(g)$ and $g = O(h)$ then $f = O(h)$.

Statement is true

Proof:

Consider the following functions

$$f(n) = n$$
$$g(n) = n^2$$
$$h(n) = n^3$$

Now, considering that $f(n) = O(g(n))$ and $g(n) = O(h(n))$ we see that the rate of growth of time complexity (TC) for $f(n)$ is less than $g(n)$ and the rate of growth of TC of $g(n)$ is less than $h(n)$ such that the growth rates can be represented as follows:

$$f(n) < g(n) < h(n)$$

Thus, $f(n) < h(n)$ and $f = O(h)$ is true Δ.

d) If $f = \Omega(g)$ and $h = \Omega(g)$, then $f = \Omega(h)$

Statement NOT true. Because f might not always be asymptotically greater than or equal to h.

Proof by Contradiction

Consider the following functions:

$$f(n) = n^2$$
$$h(n) = n^3$$
$$g(n) = n$$

Thus as provided/given, $f = \Omega(g)$, $h = \Omega(g)$ But $f \neq \Omega(h)$ since there does NOT exist a constant $> 0 \in \mathbb{N}$ and or threshold where $f(n) \geq h(n) \cdot c$ for all $n > n_0$.

$$n^2 \not\geq c \cdot n^3 \Rightarrow \Leftarrow$$

Thus statement is Not true Δ

Eaton B. Cruz

Q3)

Prove or disprove. If true, Provide a proof. Otherwise Provide a counter example and justify why the counterexample works.

a) False. Let $f(n) = n^2+10n$ and $g(n) = n^3$. $f(n) = O(g(n))$ iff $\exists c > 0$ ($c \in \mathbb{N}$) and a threshold $n_0 > 0$ such that

$$f(n) \leq cg(n) \text{ for all } n \geq n_0.$$

Let $c = 10$ and $n_0 = 0$. such that $n^2+10n \leq 10n^3 \ \forall n \geq 0$.

Thus by def, $f(n) = O(g(n))$.

Now, $g = O(f(n)) \Rightarrow \Leftarrow$. There does not exist a constant c or a threhold n_0 such that

$$g(n) \leq cf(n) \text{ or } n^3 < c \cdot n^2+10n \text{ for all } n \geq n_0.$$

Thus, the statement is false, by definition of $f(n) = O(g(n))$.

b) If $f = \Omega(n)$ and $g = \Omega(n)$, then $f+g = \Omega(n)$

True.

Proof

Let $f(n) = n^3$ $g(n) = n^2$, $h(n) = n$ where $f(n) = \Omega(h(n))$ $g(n) = \Omega(h(n))$

$(f+g)\,n = n^3+n^2$. If $(f+g)\,n = \Omega(h(n))$ then $\exists c > 0$ ($c \in \mathbb{N}$) and threshold such that $(f+g)\,n \geq c \cdot h(n)$. Let $c = 10$, $n_0 = 0$ s.t.

$$n^3+n^2 \geq 10n \checkmark \text{ statement holds true thus}$$

$$(f+g)\,n = \Omega(h(n)) \triangle$$

Q4)

4a) Write the pseudo code for selection_Sort.

Assume index starts at 1 for simplicity

Selection_Sort (A[1. . . . n])

for i = n. to 0 // recall end point is exclusive so this will // iterate from n to 1 since 0 ≠ included

(1) max = A[i]
(2) max_index = i
(3) j = i-1
(4) aux = A[j]
(5) 5.1 while (j > 0) //will always run to get largest element
5.2 If (A[j] > max) //only update if larger: stable sort
5.2.1 max = A[j]
5.2.2 max_index = j
5.2.3 j = j-1
(6) 6.1 If (max ≠ A[i]) //only if we found larger element (swap only if nec.)
6.2 A[max_index] = A[i]
6.3 A[i] = max

4b) Find TC in best / worst case scenarios. Justify.

Worst case: have to swap every iteration. Labels for commands have been provided above such that we can analyze the

Extra Space Needed

1	2	3	4	5	6
Constant	Constant	Constant	Constant	t_i	t_i

t_i → time needed to run command 5 on i^{th} for loop iteration

$$c + c + c + c + t_i + t_i = 4c + 2t_i$$

Thus total time complexity $= \sum_{i=1}^{n} (4c + 2t_i) = \sum_{i=1}^{n} 4c + \sum_{i=1}^{n} 2t_i = 4\sum_{i=1}^{n} c + 2\sum_{i=1}^{n} t_i$
$= (4c)n + 2\sum_{i=1}^{n} t_i$. Let $t_i = c'$ for $i = 1 \ldots n$ s.t. $TTC = 4cn + 2c'n \Rightarrow n(4c + 2c')$.

Continued on Next Page.

Ezra Berry

Q4

4b) $TTC = n(4c + 2c^2) = 4cn + 2c^2n$

worst case: $t_i = c^2 i$ for $i = 1 \ldots n$

$$= 4cn + 2\sum_{i=1}^{n} c^2 i$$

$$= 4cn + 2c^2 \sum_{i=1}^{n} i \leftarrow \quad 1 + \ldots n = \frac{n(n+1)}{2}$$

$$= 4cn + 2c^2 \cdot \frac{n(n+1)}{2}$$

$$= 4cn + 2c^2 \frac{(n^2+n)}{2}$$

$$= 4cn + \frac{2c^2n^2 + 2c^2n}{2} = c^2n^2 + c^2n + 4cn$$

$c^2(n^2+n) + 4cn \rightarrow$ TC is quadratic in the input size

Best case: already sorted. While loop runs but if statements dont: $4c + t_i$

$$4cn + \sum_{i=1}^{n} t_i, \quad t_i = c^2 \text{ for } i = 1 \ldots n$$

$$= 4cn + c^2 n$$

$n(\underbrace{4c + c^2}_{\text{a constant}}) \rightarrow$ linear w/ input size

4c) This variant is more time-efficient because it cuts down the input size by 2 since we are finding the two largest elements and placing them at the end thus it will take less time to sort our input array.

Q5)

Erin Bling

a) when do these two functions return TRUE?

Compare 1 will return true when at every index the values A[i] + C[i] < B[j]. Note that i=j.

Compare 2 will return true when the aux variable which at the end turns out to be the greates Possible value that A[i] + C[i] can amount to So for every index this functions checks to see the greatest value that A[i] + C[i] can equal

Then it will check if aux is greater than any element in B.

Thus, both functions will return True if at every index the sum A[i] + C[i] is less than B[j], knowing i=j and the the greatest possible value that A[i] + C[i] can amount to (aux) is less than every value of B.

The following arrays with n elements will satisfy these conditions and return true for both functions

A = [1 2 3 4]
B = [2 3 4 5]
C = [10 20 30 40]

b) worst Running Time for each function

Compare 1: $O(n^2)$ → both for loops run

compare 2: $O(n)$ → both for loops run but only one goes through whole input array

c) Best Running Time:

Compare 1: $O(n^2)$ → both for loops always run

Compare 2: $O(n)$ → One for loops runs through whole array, it always runs since there is no criteria for it not to (only if statements)

only if statements might not run but criteria for the for loops runs from 1 index 1 to n